MA 641: TIME SERIES ANALSIS 1

Project Report

Time Series Analysis and Forecasting of Seasonal and Non-Seasonal Data

Name: Sathvik Vadlapatla **Year:** 2024

1. Introduction

Time series analysis and forecasting are required for predicting patterns, trends, and fluctuations in data over a period of time. Investors, Businesses, and Policymakers depend on such analyses to make decisions. This report combines the analysis of two different datasets:

- **Seasonal Data:** Electric car sales in USA, which shows monthly patterns over a period of time.
- **Non-Seasonal Data:** Yahoo daily stock closing prices over five years, which shows trends and volatility.

This project uses Box-Jenkins approach for time series modeling and forecasting. By using ARIMA and SARIMA models the projects aims to:

- Analyze the patterns and trends in the data.
- Provide the accurate forecasts for decision-making.
- Gives the insights and recommendations based on findings.

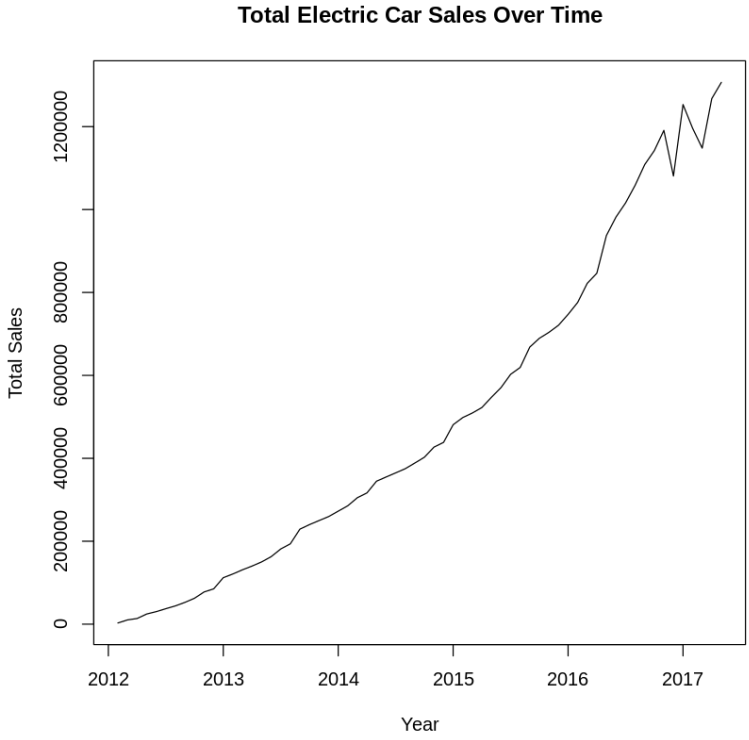
2. Data

2.1 Seasonal Data: Electric Car Sales

- Source: Extracted from Flourish Visualizations (Kaggle Dataset)
  Description:
  - Monthly sales data for electric cars models in USA.
  - From several years with each month containing aggregated sales.

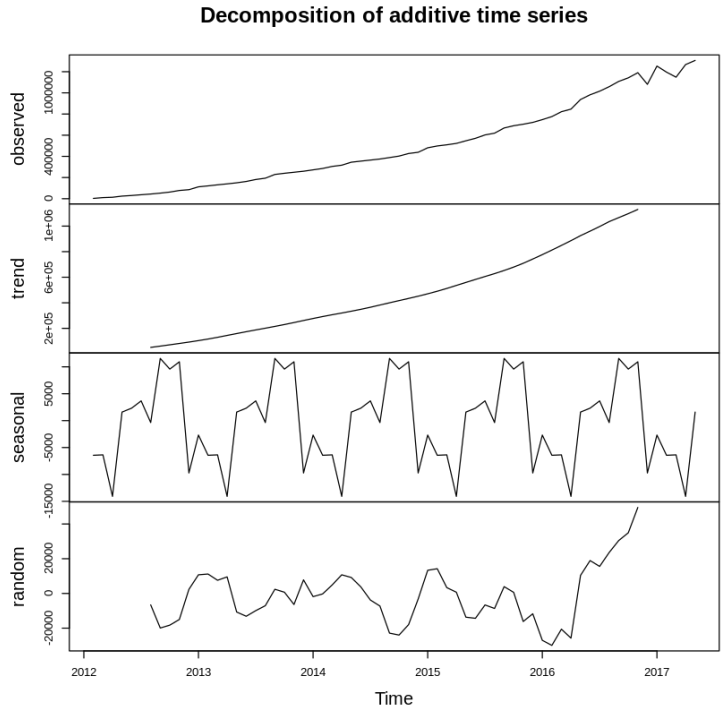
Descriptive Analysis

1. Visualization:
   Below a time series plot showing a clear upward trend with seasonal patterns which shows periodic increase during specific months.



2. Seasonality:

Decomposition of the time series shows the components for trends, seasonality, and residuals.



3. Issues:

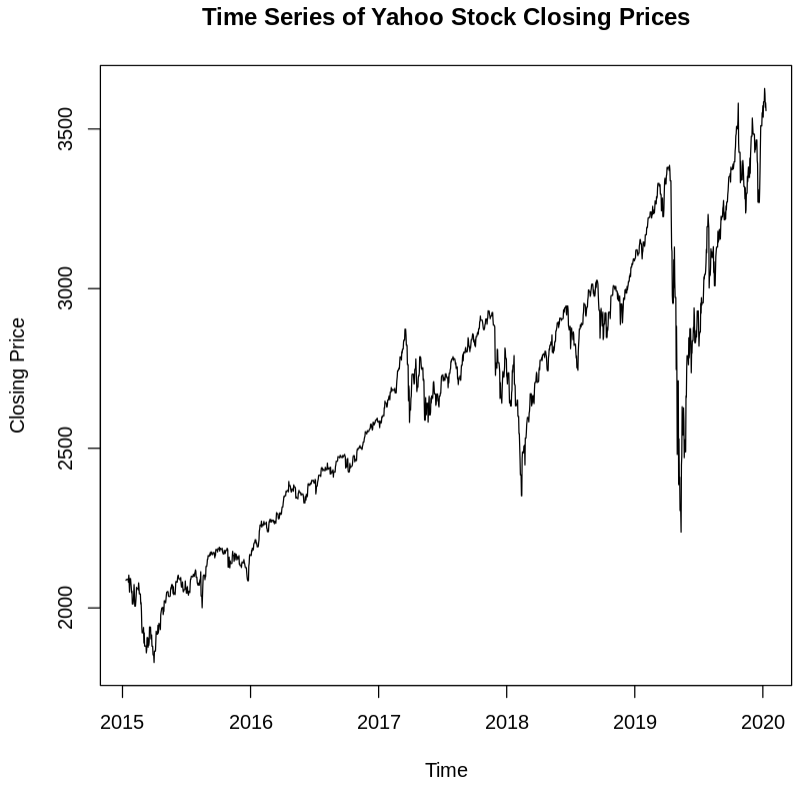
   Some missing values are identified and imputed using forward filling methods to ensure continuity in analysis.

2.2 Non-Seasonal Data: Yahoo Daily Stock Prices

- Source: Yahoo Finance(Kaggle Dataset)
- Description:
  - Daily stock prices for a five year time period.
  - Variables containing High, Low, Open, Close, Volume, and Adjusted Close.

Descriptive Analysis

- Visualization:



- Stationarity Check:
  The ADF test result shows a p-value of 0.01 which is small than 0.05 which tells that we can reject null hypothesis of non-stationarity. So this confirms that the series is stationary.
- Issues:

  No missing values or outliers are detected.

3. Box-Jenkins Models

3.1 Seasonal Data: Electric Car Sales

Model Selection

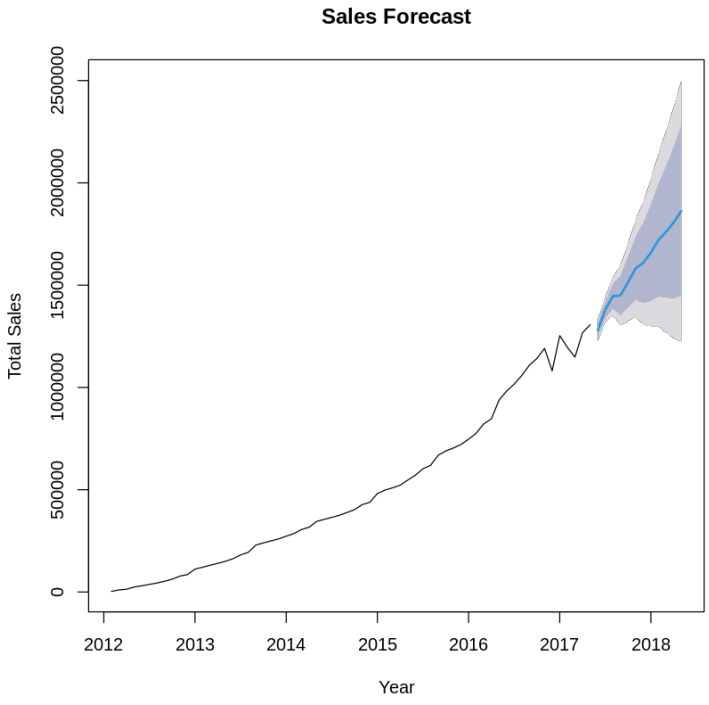
- Seasonal data requires differencing to ensure stationarity.
- After checking for stationarity and decomposition. A ARIMA(2,2,2) model is selected.

Model Fit

- AIC: 1453.38
- RMSE: 26061.1
- Residual Analysis:
    - Residual behaves like white noise with no significant autocorrelation.
    - The Ljung-Box test with p-value >0.05 indicates a good model fit.

Forecasting

- Predicted monthly sales of the next 12 months.



- Point Forecast: The blue line shows the predicted sales for each moth.
    - Example: Predicted sales for Jun 2017 is 1,279,077 units.
- Confidence Intervals:
    - Light Shaded Area(80%): Shows a higher probability of true sales value falling this range.

    - Darker Shaded Area(95%): Shows wider range giving more conservative estimation of uncertainty.
- Insights:
    - We can see that sales are expected to grow steadily over time.
    - Range of uncertainty increases for forecast into the future as indicated by increasing spread of confidence intervals.

3.2 Non Seasonal Data: Yahoo Stock price

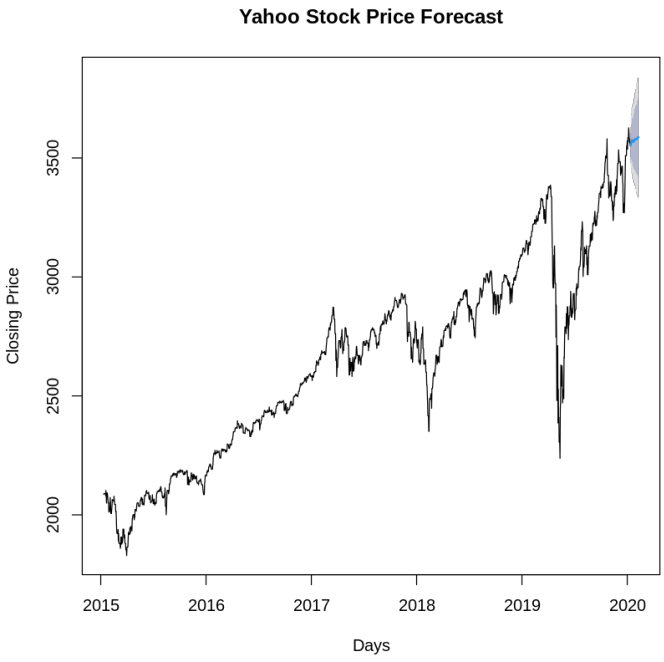
Model Selection

- Done by using `auto.arima()` function. A ARIMA(5,1,2) with drift model has been chosen based on AIC minimization.

Model Fit

- AIC: 17173.57
- RMSE: 26.67
- MAPE: 0.529%
- Residual Analysis:
    - Residuals are approximately normally distributed with a minor autocorrelation.
    - The Ljung-Box test with p-value < 0.05 shows slight residual autocorrelation.

Forecasting

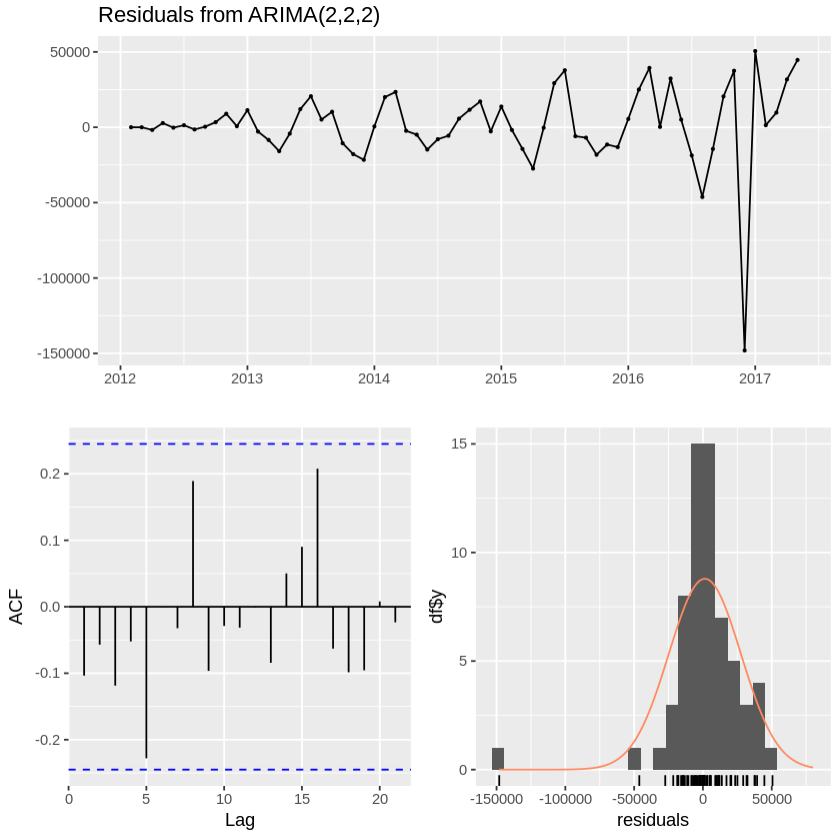


- Point Forecast:
    - Blue line in plot represents the predicted stock price for next 30 days.
    - For example the forecasted stock price on day 1 is 3556.89.
- Confidence Intervals:
    - Lighter shaded area(80%): Indicates where stock price is likely to fall with 80 probability.
    - Darker shaded area(95%): Shows the wider range with 95% certainty.
- Steady trend:
    - We can see that forecast predicts steady increase in stock price with very few fluctuations.

4. Statistical Conclusions

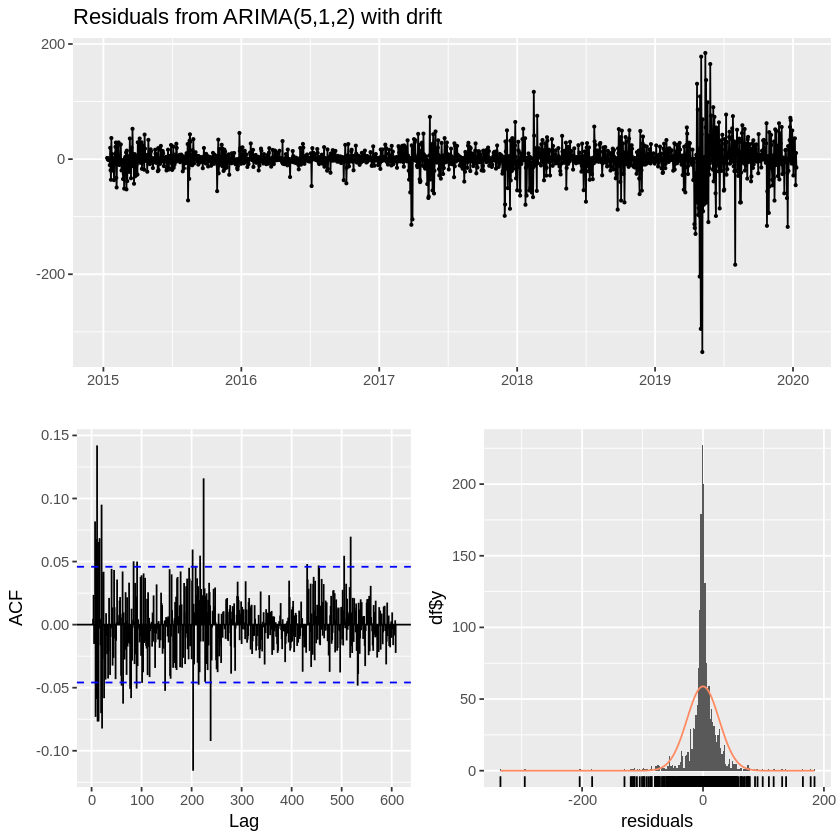
Seasonal Data: Electric Car Sales

- Model Performance:
    - ARIMA(2,2,2) is best fitting model for the data and captures the underlying trend.
    - Metrics like AIC and RMSE shows the model accuracy.
- Forecast Implications:
    - The forecast predicted steady growth in the electric car sales over the next year.
    - We can see the seasonal peaks aligned with historical trends.



Non Seasonal Data: Yahoo Stock Price

- Model Performance:
  - ARIMA(5,1,2) model captures the data nicely with low RMSE and MAPE while training.
  - Residual diagnostics confirms that model is appropriate for forecasting.
- Forecast Implications:
  - Stock price is expected to remain stable with little bit upward trend for the 30 days
  - Confidence intervals shows the potential volatility but the overall trend is upward.



5. Contextual Conclusions

- **Electric Car Sales:**
  - The growth trajectory highlights and shows increasing adoption of electric vehicles in USA.
  - Manufacturers also can you the forecasts to plan the production and inventory.
- **Yahoo Stock Price:**
  - The steady upward trend shows the positive market performance.
  - The investors should also consider potential volatility when making short term decisions.

Appendix

Code and Outputs

Seasonal Data:

```
# Loading required libraries
library(tidyverse)
library(lubridate)

# Loading the dataset
data <- read.csv("Electric Car Sales by Model in USA.csv")

# Inspecting structure and first few rows of the dataset
str(data)
head(data)
```

── **Attaching core tidyverse packages** ──────────────────────────────── tidyverse 2.0.0 ──
✓ dplyr 1.1.4 ✓ readr 2.1.5
✓ forcats 1.0.0 ✓ stringr 1.5.1
✓ ggplot2 3.5.1 ✓ tibble 3.2.1
✓ lubridate 1.9.3 ✓ tidyr 1.3.1
✓ purrr 1.0.2
── **Conflicts** ── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag() masks stats::lag()
i Use the conflicted package (<*http://conflicted.r-lib.org/*>) to force all conflicts to become errors
'data.frame': 57 obs. of 99 variables:
$ Make : chr "Chevrolet" "Toyota" "Nissan" "Tesla" ...
$ Model : chr "Volt" "Prius PHV" "Leaf" "Model S" ...
$ Logo : chr "https://www.carlogos.org/logo/Chevrolet-logo-2013-2560x1440.png" "https://www.carlogos.org/logo/Toyota-logo-1989-2560x1440.png" "https://www.carlogos.org/logo/Nissan-logo-2013-1440x900.png" "https://www.carlogos.org/logo/Tesla-logo-2003-2500x2500.png" ...
$ janv.12 : num 603 0 676 0 0 2 36 0 0 0 ...
$ Feb.2012: num 1626 21 1154 NA NA ...
$ mars.12 : num 3915 912 1733 NA NA ...
$ Apr.2012: num 5377 2566 2103 NA NA ...

$ May.2012: num 7057 3652 2613 NA NA ...
$ juin.12 : num 8817 4347 3148 12 NA ...
$ juil.12 : num 10666 5035 3543 31 NA ...
$ Aug.2012: num 13497 6082 4228 74 NA ...
$ sept.12 : num 16348 7734 5212 160 NA ...
$ oct.12 : num 19309 9623 6791 460 144 ...
$ nov.12 : num 20828 11389 8330 860 1403 ...
$ Dec.2012: num 23461 12750 9819 2650 2374 ...
$ janv.13 : num 24601 13624 10469 3850 2712 ...
$ Feb.2013: num 26227 14317 11122 5250 3046 ...
$ mars.13 : num 27705 15103 13358 7550 3540 ...
$ Apr.2013: num 29011 15702 15295 9650 3951 ...
$ May.2013: num 30618 16380 17433 11350 4401 ...
$ juin.13 : num 33316 16964 19658 12700 4856 ...
$ juil.13 : num 35104 17781 21522 14000 5289 ...
$ Aug.2013: num 38455 19572 23942 15300 5910 ...
$ sept.13 : num 40221 20724 25895 16800 6668 ...
$ oct.13 : num 42243 22819 27897 17600 7760 ...
$ nov.13 : num 44163 23919 29900 18800 8701 ...
$ Dec.2013: num 46555 24838 32429 20300 9528 ...
$ janv.14 : num 47473 25641 33681 21100 9999 ...
$ Feb.2014: num 48683 26682 35106 22189 10551 ...
$ mars.14 : num 50161 28134 37613 23489 11161 ...
$ Apr.2014: num 51709 29875 39701 24589 11686 ...
$ May.2014: num 53393 32567 42818 25589 12468 ...
$ juin.14 : num 55170 34138 45165 27089 13456 ...
$ juil.14 : num 57190 35509 48184 27889 14287 ...
$ Aug.2014: num 59701 36327 51370 28489 15337 ...
$ sept.14 : num 61095 36680 54251 30989 16014 ...
$ oct.14 : num 62534 37159 56840 32289 16658 ...
$ nov.14 : num 63870 37610 59527 33489 17302 ...
$ Dec.2014: num 65360 38102 62629 36989 17961 ...
$ janv.15 : num 65902 38503 63699 38089 18356 ...
$ Feb.2015: num 66595 38900 64897 39239 18854 ...
$ mars.15 : num 67234 39373 66714 41689 19569 ...
$ Apr.2015: num 68139 39801 68267 43389 20122 ...
$ May.2015: num 69757 40528 70371 45789 20837 ...
$ juin.15 : num 70982 40992 72445 48589 21504 ...
$ juil.15 : num 72295 41576 73619 50189 22197 ...
$ Aug.2015: num 73675 41920 75012 51489 22920 ...

$ sept.15 : num 74624 42136 76259 53989 23639 ...
$ oct.15 : num 76659 42227 77497 55889 24334 ...
$ nov.15 : num 78639 42271 78551 58591 24973 ...
$ Dec.2015: num 80753 42293 79898 62191 25552 ...
$ janv.16 : num 81749 42303 80653 63041 25902 ...
$ Feb.2016: num 82875 42309 81583 64591 26392 ...
$ mars.16 : num 84740 42316 82829 68581 27002 ...
$ Apr.2016: num 86723 42320 83616 69381 27609 ...
$ May.2016: num 88624 42324 84595 70581 28147 ...
$ juin.16 : num 90561 42335 85691 74281 28777 ...
$ juil.16 : num 92967 42339 86754 76235 29532 ...
$ Aug.2016: num 95048 42341 87820 79087 30239 ...
$ sept.16 : num 97079 42345 89136 83437 30928 ...
$ oct.16 : num 99270 42345 90548 84137 31499 ...
$ nov.16 : num 101801 42345 92005 85237 32220 ...
$ Dec.2016: num 105492 42345 93904 91087 33509 ...
$ janv.17 : num 107103 42345 94676 91987 33982 ...
$ Feb.2017: num 108923 42345 95713 93737 34621 ...
$ mars.17 : num 111055 42345 97191 97187 35283 ...
$ Apr.2017: num 112862 42345 98254 98312 36003 ...
$ May.2017: num 114679 42345 99646 99932 36953 ...
$ juin.17 : num 116424 42345 101152 102282 37889 ...
$ juil.17 : num 117942 42345 102435 103707 38733 ...
$ Aug.2017: num 119387 42345 103589 105857 39438 ...
$ sept.17 : num 120840 42345 104644 110717 40121 ...
$ oct.17 : num 122202 42345 104857 111837 40690 ...
$ nov.17 : num 123904 42345 105032 113172 41213 ...
$ Dec.2017: num 125841 42345 105134 118147 41649 ...
$ janv.18 : num 126554 42345 105284 118947 41883 ...
$ Feb.2018: num 127537 42345 106179 120072 42025 ...
$ mars.18 : num 129319 42345 107679 123447 42130 ...
$ Apr.2018: num 130644 42345 108850 124697 42187 ...
$ May.2018: num 132319 42345 110426 126217 42205 ...
$ juin.18 : num 133655 42345 111793 128967 42211 ...
$ juil.18 : num 135130 42345 112942 130167 42215 ...
$ Aug.2018: num 136955 42345 114257 132792 42219 ...
$ sept.18 : num 139084 42345 115820 136542 42231 ...
$ oct.18 : num 140559 42345 117054 137892 42231 ...
$ nov.18 : num 143089 42345 118182 140642 42231 ...
$ Dec.2018: num 144147 42345 119849 143892 42231 ...

$ janv.19 : num 144822 42345 120566 144617 42231 ...
$ Feb.2019: num 145437 42345 121220 145242 42231 ...
$ mars.19 : num 146667 42345 122534 147517 42231 ...
$ Apr.2019: num 147072 42345 123485 148342 42231 ...
$ May.2019: chr "147,48" "42345" "124,701" "149,367" ...
$ juin.19 : chr "147,813" "42345" "125,857" "151,117" ...
$ juil.19 : chr "148,063" "42345" "126,795" "152,092" ...
$ Aug.2019: chr "148,337" "42345" "127,912" "153,142" ...
$ sept.19 : chr "148,687" "42345" "128,96" "154,242" ...
$ oct.19 : chr "148,757" "42345" "129,847" "154,992" ...
$ nov.19 : chr "148,907" "42345" "130,987" "156,492" ...
$ Dec.2019: chr "149,057" "42345" "132,214" "157,992" ...

```
# Converting all columns related to sales into numeric,
replacing commas and characters
sales_cols <-
grep("janv.|Feb.|mars.|Apr.|May.|juin.|juil.|Aug.|sept.|oct.|nov
.|Dec.", colnames(data), value = TRUE)

# Removing commas and convert to numeric
data[sales_cols] <- lapply(data[sales_cols], function(x)
as.numeric(gsub(",", "", x)))

# Reshaping the data into long format
sales_long <- data %>%
  pivot_longer(cols = all_of(sales_cols),
               names_to = "Month",
               values_to = "Sales")

# Summarizing total sales by month
monthly_sales <- sales_long %>%
  group_by(Month) %>%
  summarize(Total_Sales = sum(Sales, na.rm = TRUE))

# Showing the first few rows of the summarized data
head(monthly_sales)
```

A tibble: 6 × 2

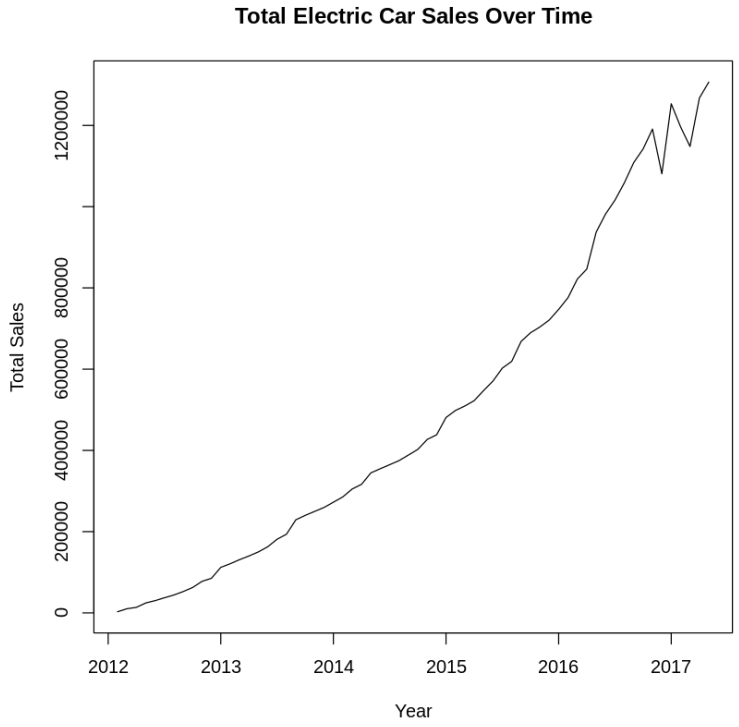
Month	Total_Sales
<chr>	<dbl>
Apr.2012	10263
Apr.2013	77708

Apr.2014 181369
Apr.2015 304995
Apr.2016 427023
Apr.2017 602553

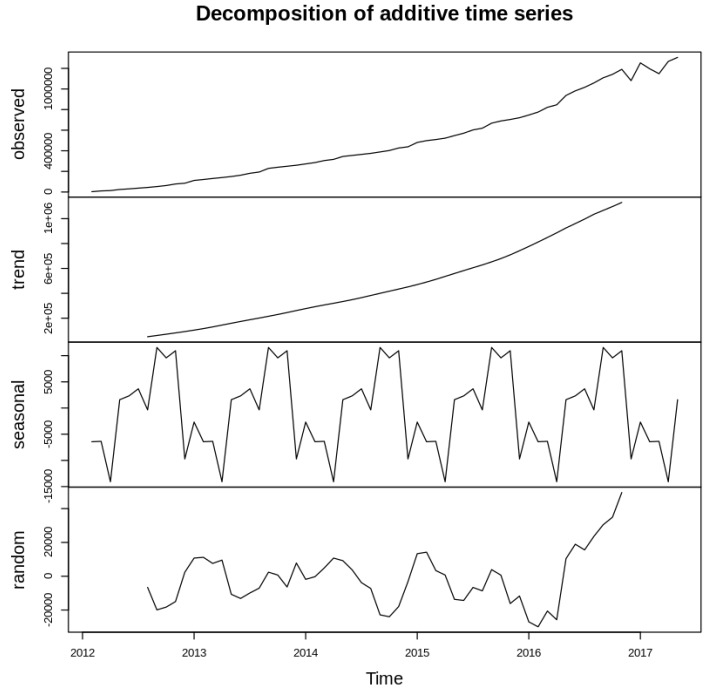
```
# Checking unique values in the Month column
unique(monthly_sales$Month)
```

[1] "2012-02-01 UTC" "2012-04-01 UTC" "2012-05-01 UTC" "2012-08-01 UTC"
[5] "2012-09-01 UTC" "2012-10-01 UTC" "2012-11-01 UTC" "2012-12-01 UTC"
[9] "2013-02-01 UTC" "2013-04-01 UTC" "2013-05-01 UTC" "2013-08-01 UTC"
[13] "2013-09-01 UTC" "2013-10-01 UTC" "2013-11-01 UTC" "2013-12-01 UTC"
[17] "2014-02-01 UTC" "2014-04-01 UTC" "2014-05-01 UTC" "2014-08-01 UTC"
[21] "2014-09-01 UTC" "2014-10-01 UTC" "2014-11-01 UTC" "2014-12-01 UTC"
[25] "2015-02-01 UTC" "2015-04-01 UTC" "2015-05-01 UTC" "2015-08-01 UTC"
[29] "2015-09-01 UTC" "2015-10-01 UTC" "2015-11-01 UTC" "2015-12-01 UTC"
[33] "2016-02-01 UTC" "2016-04-01 UTC" "2016-05-01 UTC" "2016-08-01 UTC"
[37] "2016-09-01 UTC" "2016-10-01 UTC" "2016-11-01 UTC" "2016-12-01 UTC"
[41] "2017-02-01 UTC" "2017-04-01 UTC" "2017-05-01 UTC" "2017-08-01 UTC"
[45] "2017-09-01 UTC" "2017-10-01 UTC" "2017-11-01 UTC" "2017-12-01 UTC"
[49] "2018-02-01 UTC" "2018-04-01 UTC" "2018-05-01 UTC" "2018-08-01 UTC"
[53] "2018-09-01 UTC" "2018-10-01 UTC" "2018-11-01 UTC" "2018-12-01 UTC"
[57] "2019-02-01 UTC" "2019-04-01 UTC" "2019-05-01 UTC" "2019-08-01 UTC"
[61] "2019-09-01 UTC" "2019-10-01 UTC" "2019-11-01 UTC" "2019-12-01 UTC"
[65] NA

```
# Removing rows with NA in Month
monthly_sales <- monthly_sales %>%
  filter(!is.na(Month))

# Converting to a time series object
sales_ts <- ts(monthly_sales$Total_Sales,
               start = c(year(min(monthly_sales$Month)),
month(min(monthly_sales$Month))),
               frequency = 12)

# Plotting the time series
plot(sales_ts,
     main = "Total Electric Car Sales Over Time",
     xlab = "Year",
     ylab = "Total Sales")
```




```
# Decomposing time series into trend, seasonality, and residuals
decomposed <- decompose(sales_ts)

# Plotting decomposed components
plot(decomposed)
```



```
library(tseries)

# Performing Augmented Dickey-Fuller test
adf_test <- adf.test(sales_ts)

# Printing test results
adf_test
```

Registered S3 method overwritten by 'quantmod':
method from
as.zoo.data.frame zoo

Augmented Dickey-Fuller Test

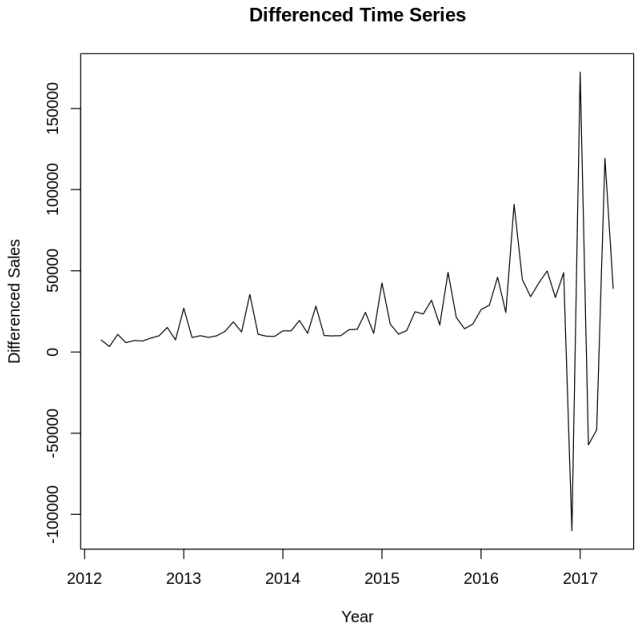
data: sales_ts
Dickey-Fuller = -0.87145, Lag order = 3, p-value = 0.9506
alternative hypothesis: stationary

- ADF test shows high p-valve of 0.9506 which tells that null hypothesis i.e. time series is non stationary and can't be rejected. This tells that the time series is non stationary so we need to transform it to get stationarity before fitting model.

```
# first-order differencing
diff_sales_ts <- diff(sales_ts)

# Performing ADF test on differenced data
adf_test_diff <- adf.test(diff_sales_ts)

# Printing the test results
adf_test_diff

# Plotting the differenced time series
plot(diff_sales_ts,
     main = "Differenced Time Series",
     xlab = "Year",
     ylab = "Differenced Sales")
```

Augmented Dickey-Fuller
Test data: diff_sales_ts

Dickey-Fuller = -2.4552, Lag order = 3, p-value = 0.3906
alternative hypothesis: stationary



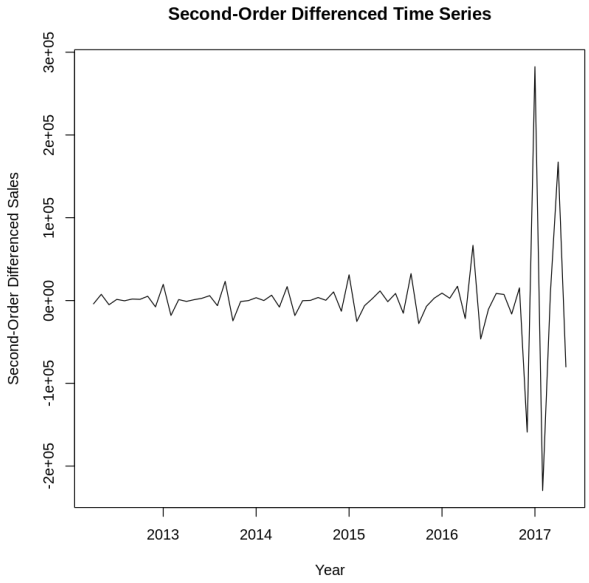
- ADF test on differenced series also shows high p-valve of 0.3906 showing that the series might still not be stationary. So will try second order differencing

```
# Second-order differencing
diff2_sales_ts <- diff(diff_sales_ts)

# Performing ADF test on second-order differenced data
adf_test_diff2 <- adf.test(diff2_sales_ts)

# Printing test results
adf_test_diff2

# Plotting second-order differenced time series
plot(diff2_sales_ts,
     main = "Second-Order Differenced Time Series",
     xlab = "Year",
     ylab = "Second-Order Differenced Sales")
```

Warning message in adf.test(diff2_sales_ts):
“p-value smaller than printed p-value”

Augmented Dickey-Fuller Test

data: diff2_sales_ts
Dickey-Fuller = -5.3439, Lag order = 3, p-value = 0.01
alternative hypothesis: stationary



- Second order differencing has made the series stationary.

```
library(forecast)

# Fitting ARIMA model to original time series
arima_model <- auto.arima(sales_ts, seasonal = TRUE)

# Summary of the ARIMA model
summary(arima_model)
```

Series: sales_ts
ARIMA(2,2,2)

Coefficients:
ar1 ar2 ma1 ma2
-0.6130 -0.6427 -0.7674 0.8185
s.e. 0.1347 0.1319 0.1224 0.1190
sigma^2 = 749440788: log likelihood = -721.69
AIC=1453.38. AICc=1454.45 BIC=1464.01
Training set error measures:

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1
Training set	1402.886	26061.1	15467.31	0.4298246	3.811731	0.06094694	-0.1035427

```
# Forecasting the next 12 months
sales_forecast <- forecast(arima_model, h = 12)

# Plotting the forecast
plot(sales_forecast,
     main = "Sales Forecast",
     xlab = "Year",
     ylab = "Total Sales")

# Printing the forecasted values
sales_forecast
```

	Point Forecast	Lo 80	Hi 80	Lo 95	Hi 95
Jun 2017	1279077	1243993	1314161	1225421	1332733
Jul 2017	1380524	1339253	1421796	1317405	1443644
Aug 2017	1445756	1385249	1506263	1353218	1538293
Sep 2017	1450301	1353233	1547370	1301848	1598754
Oct 2017	1515326	1390232	1640419	1324012	1706640
Nov 2017	1582279	1426047	1738511	1343343	1821215
Dec 2017	1609179	1412462	1805895	1308327	1910031
Jan 2018	1659393	1423011	1895775	1297878	2020908
Feb 2018	1721058	1444162	1997953	1297583	2144532
Mar 2018	1760719	1438074	2083363	1267276	2254161
Apr 2018	1806509	1436599	2176418	1240781	2372236
May 2018	1862684	1444685	2280683	1223409	2501958

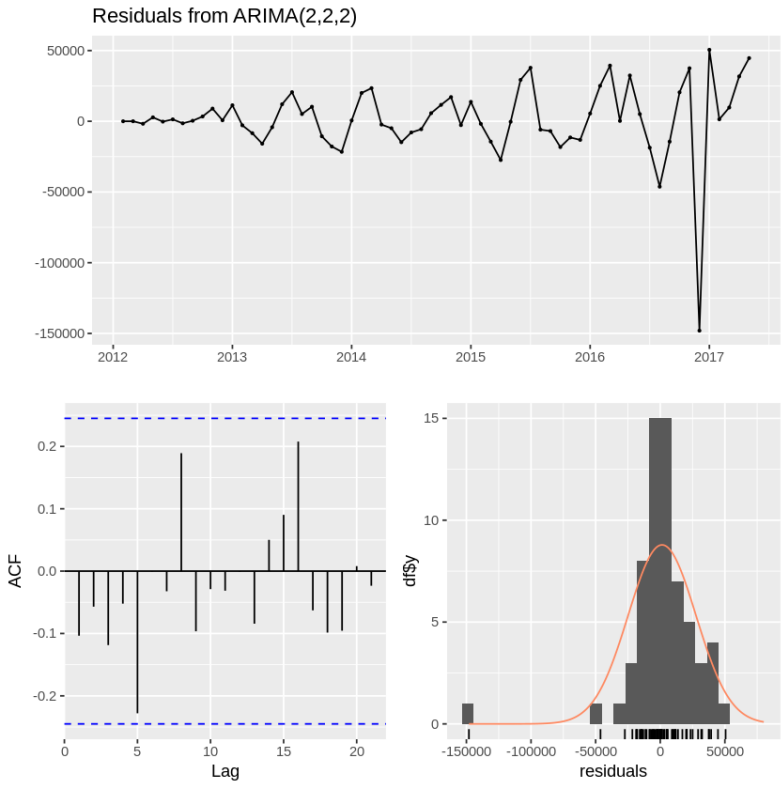
```
# Checking residual diagnostics
checkresiduals(arima_model)

# Plotting the residuals
residuals_plot <- residuals(arima_model)
plot(residuals_plot,
     main = "Residuals of the ARIMA Model",
     xlab = "Time",
     ylab = "Residuals")

# Performing Ljung-Box test on residuals
Box.test(residuals_plot, lag = 10, type = "Ljung-Box")
```

Ljung-Box test

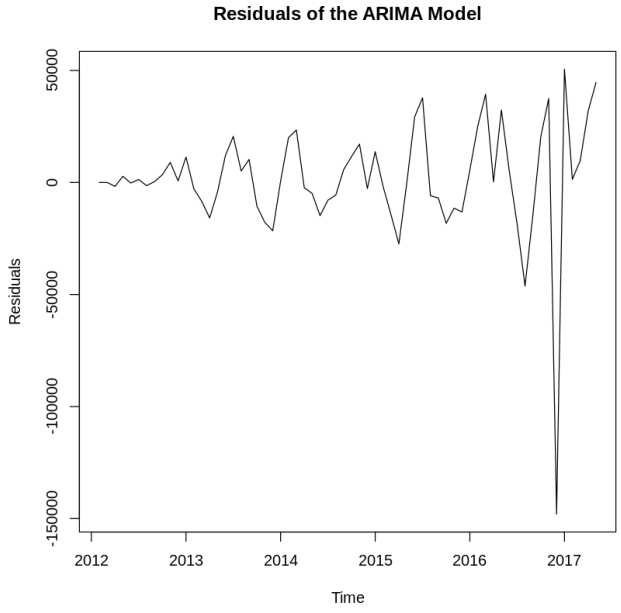
data: Residuals from ARIMA(2,2,2)

Q* = 10.047, df = 9, p-value = 0.3467

Model df: 4. Total lags used: 13



Box-Ljung test
data: residuals_plot
X-squared = 9.3807, df = 10, p-value = 0.4964



Non Seasonal Data:

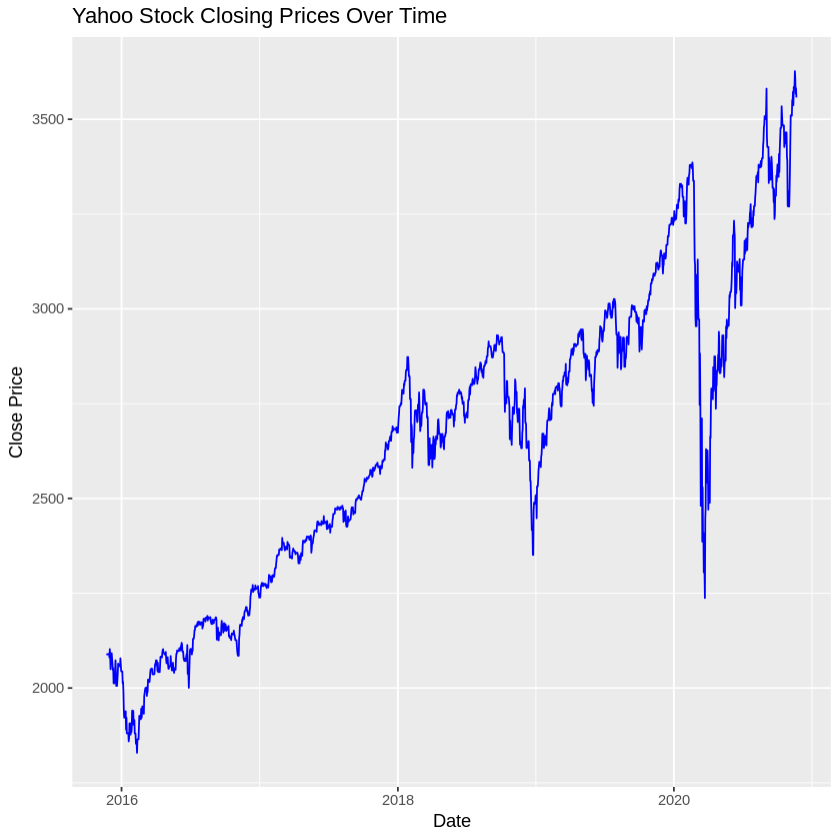
```
library(tidyverse)
library(lubridate)

# Loading dataset
stock_data <- read.csv("yahoo_stock.csv")

# Inspecting structure and first few rows of the dataset
str(stock_data)
head(stock_data)
```

'data.frame': 1825 obs. of 7 variables:
$ Date : chr "2015-11-23" "2015-11-24" "2015-11-25" "2015-11-26" ...
$ High : num 2096 2094 2093 2093 2093 ...
$ Low : num 2081 2070 2086 2086 2084 ...
$ Open : num 2089 2084 2089 2089 2089 ...
$ Close : num 2087 2089 2089 2089 2090 ...
$ Volume : num 3.59e+09 3.88e+09 2.85e+09 2.85e+09 1.47e+09 ...
$ Adj.Close: num 2087 2089 2089 2089 2090 ...

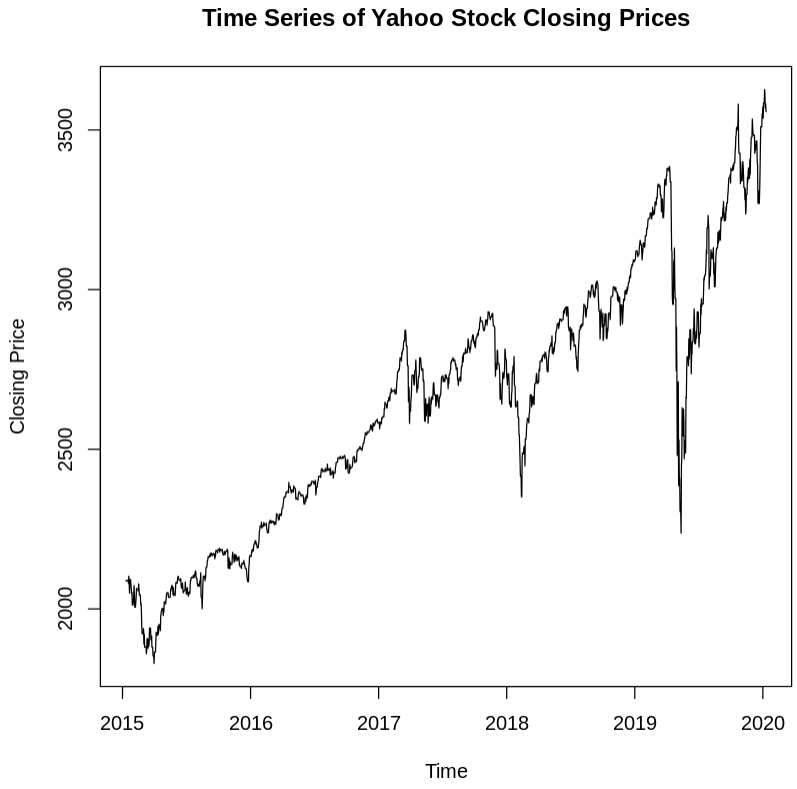
A data.frame: 6 × 7

	Date	High	Low	Open	Close	Volume	Adj.Close
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2015-11-23	2095.61	2081.39	2089.41	2086.59	3587980000	2086.59
2	2015-11-24	2094.12	2070.29	2084.42	2089.14	3884930000	2089.14
3	2015-11-25	2093.00	2086.30	2089.30	2088.87	2852940000	2088.87
4	2015-11-26	2093.00	2086.30	2089.30	2088.87	2852940000	2088.87
5	2015-11-27	2093.29	2084.13	2088.82	2090.11	1466840000	2090.11
6	2015-11-28	2093.29	2084.13	2088.82	2090.11	1466840000	2090.11

```
# Converting the Date column to Date format
stock_data$Date <- as.Date(stock_data$Date, format = "%Y-%m-%d")

# Plotting the closing prices over time to visualize the data
ggplot(stock_data, aes(x = Date, y = Close)) +
  geom_line(color = "blue") +
  labs(title = "Yahoo Stock Closing Prices Over Time", x =
"Date", y = "Close Price")
```




```
# Creating a time series object for the "Close" prices
close_ts <- ts(stock_data$Close,
               start = c(year(min(stock_data$Date)),
month(min(stock_data$Date))),
               frequency = 365)  # Daily data with 365
observations per year

# Plotting the time series
plot(close_ts,
     main = "Time Series of Yahoo Stock Closing Prices",
     xlab = "Time",
     ylab = "Closing Price")
```




```
# Performing Augmented Dickey-Fuller test
library(tseries)
adf_test <- adf.test(close_ts)

# Printing the test results
adf_test
```

Registered S3 method overwritten by 'quantmod':
method from
as.zoo.data.frame zoo

Warning message in adf.test(close_ts):
“p-value smaller than printed p-value”

Augmented Dickey-Fuller Test

data: close_ts
Dickey-Fuller = -4.0439, Lag order = 12, p-value = 0.01
alternative hypothesis: stationary

```
library(forecast)
```

```
# Fitting an ARIMA model
arima_model <- auto.arima(close_ts)

# Displaying the model summary
summary(arima_model)
```
Series: close_ts
ARIMA(5,1,2) with drift
Coefficients: ar1 ar2 ar3 ar4 ar5 ma1 ma2 drift
1.2534 -0.5035 -0.1928 0.0208 -0.0284 -1.4328 0.8302 0.8117
s.e. 0.0782 0.0622 0.0430 0.0392 0.0284 0.0751 0.0609 0.5512

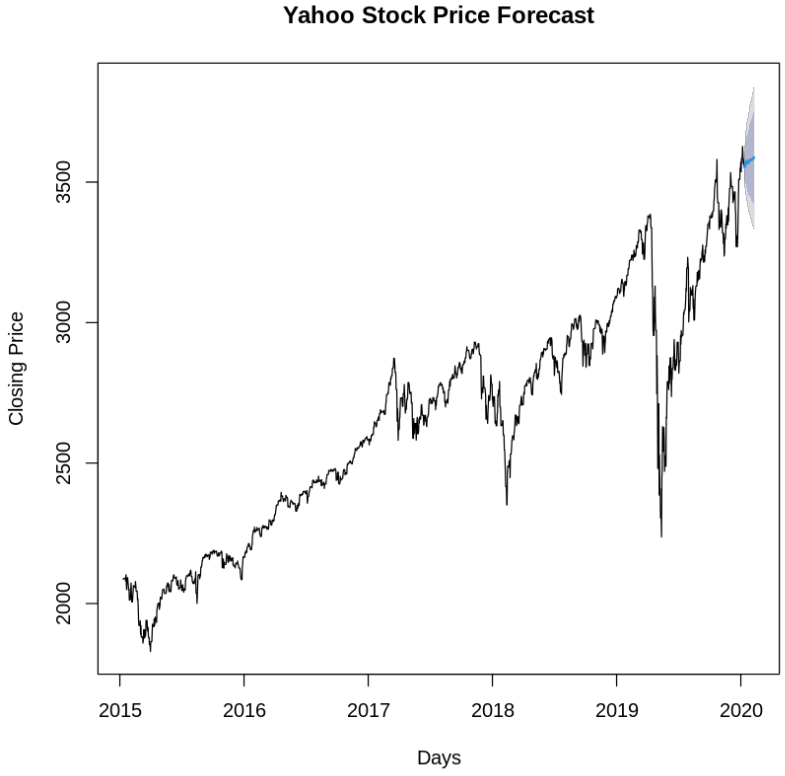
sigma^2 = 714.7: log likelihood = -8577.78
AIC=17173.57 AICc=17173.67 BIC=17223.15
Training set error measures:

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1
Training set	-0.001811586	26.66808	14.14914	-0.007803884	0.5293677	0.04701366	0.0005702465

```
# Forecasting next 30 days
stock_forecast <- forecast(arima_model, h = 30)

# Plotting the forecast
plot(stock_forecast,
     main = "Yahoo Stock Price Forecast",
     xlab = "Days",
     ylab = "Closing Price")

# Printing the forecasted values
stock_forecast
```

	Point Forecast	Lo 80	Hi 80	Lo 95	Hi 95
2020.0274	3556.892	3522.631	3591.153	3504.494	3609.290
2020.0301	3553.293	3508.972	3597.614	3485.510	3621.076
2020.0329	3555.643	3501.206	3610.080	3472.388	3638.897
2020.0356	3559.985	3496.599	3623.370	3463.045	3656.924
2020.0384	3565.980	3494.196	3637.763	3456.197	3675.763
2020.0411	3571.164	3492.152	3650.176	3450.325	3692.002
2020.0438	3574.323	3489.213	3659.433	3444.159	3704.487
2020.0466	3574.906	3484.687	3665.124	3436.929	3712.883
2020.0493	3573.413	3478.766	3668.060	3428.663	3718.164

2020.0521 3570.944 3472.229 3669.658 3419.973 3721.915
2020.0548 3568.772 3466.078 3671.466 3411.715 3725.829
2020.0575 3567.868 3461.106 3674.631 3404.590 3731.147
2020.0603 3568.624 3457.648 3679.599 3398.902 3738.346
2020.0630 3570.801 3455.524 3686.078 3394.501 3747.102
2020.0658 3573.714 3454.176 3693.252 3390.897 3756.532
2020.0685 3576.532 3452.908 3700.156 3387.465 3765.599
2020.0712 3578.585 3451.136 3706.033 3383.668 3773.501
2020.0740 3579.566 3448.571 3710.561 3379.226 3779.906
2020.0767 3579.584 3445.274 3713.895 3374.174 3784.995
2020.0795 3579.059 3441.583 3716.534 3368.808 3789.310
2020.0822 3578.530 3437.954 3719.105 3363.538 3793.522
2020.0849 3578.456 3434.781 3722.131 3358.725 3798.188
2020.0877 3579.069 3432.267 3725.871 3354.555 3803.583
2020.0904 3580.331 3430.383 3730.280 3351.005 3809.657
2020.0932 3581.988 3428.909 3735.066 3347.874 3816.101
2020.0959 3583.690 3427.542 3739.837 3344.882 3822.497
2020.0986 3585.126 3426.005 3744.247 3341.771 3828.481
2020.1014 3586.124 3424.140 3748.109 3338.390 3833.859
2020.1041 3586.689 3421.942 3751.436 3334.730 3838.648
2020.1068 3586.971 3419.538 3754.403 3330.905 3843.037



```
# Checking residual diagnostics
checkresiduals(arima_model)

# Performing Ljung-Box test
Box.test(residuals(arima_model), lag = 10, type = "Ljung-Box")
```

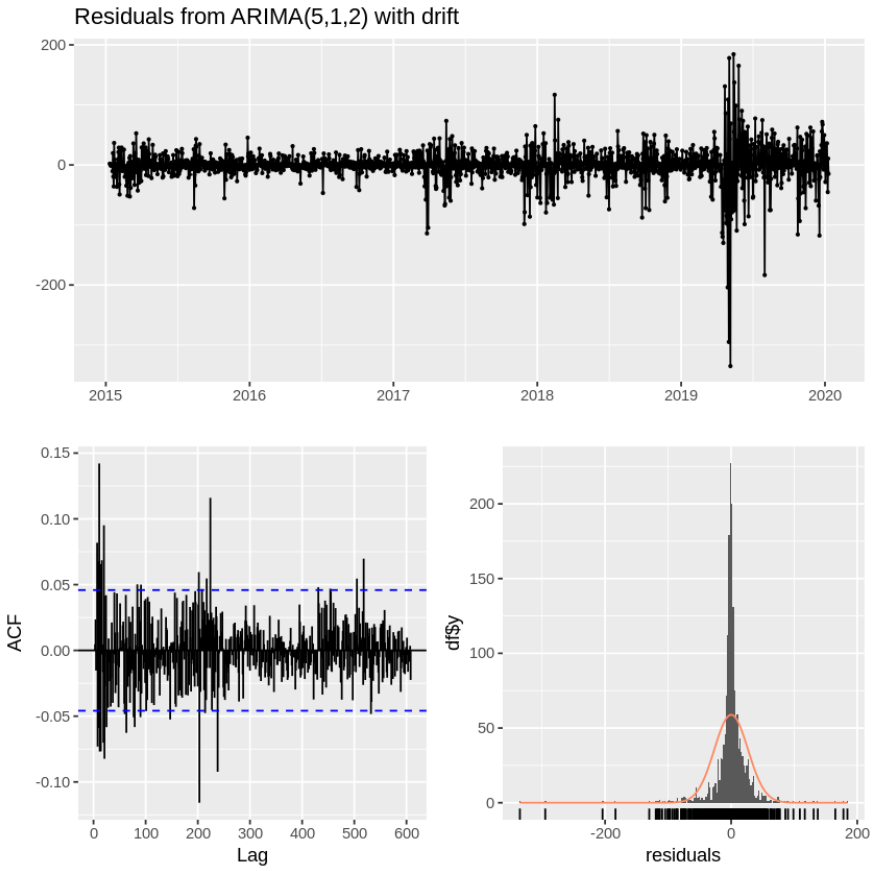
Ljung-Box test

data: Residuals from ARIMA(5,1,2) with drift
Q* = 590.61, df = 358, p-value = 1.148e-13

Model df: 7. Total lags used: 365

Box-Ljung test

data: residuals(arima_model)
X-squared = 38.439, df = 10, p-value = 3.183e-05



References

- George E. P. Box, Gwilym M. Jenkins, & Gregory C. Reinsel. Time Series Analysis: Forecasting and control.
- Flourish Visualization(Seasonal Data).
- Yahoo Finance(Non Seasonal Data).